Mathematical Formulation of the Optimization Problem

DeliveryIQ — Route Optimizer

Contents

1 Overview **2**

2 Road-Network Graph Model **2**
- 2.1 Directed Weighted Graph . . . 2
- 2.2 Largest Strongly Connected Component (LSCC) . . . 2
- 2.3 Modal Travel-Time Stamping . . . 2

3 Shortest-Path Distance Matrix **3**
- 3.1 All-Pairs Shortest Paths (Dijkstra) . . . 3
- 3.2 Penalty Sentinel . . . 4
- 3.3 Hybrid Last-Meter (Drive Mode) . . . 4

4 Single-Vehicle: Travelling Salesman Problem **4**
- 4.1 Problem Formulation . . . 4
- 4.2 Solution Methods . . . 4
  - 4.2.1 Nearest Neighbour (NN) . . . 5
  - 4.2.2 2-opt Local Search . . . 5
  - 4.2.3 Christofides Approximation . . . 5
  - 4.2.4 Genetic Algorithm (Order-Crossover, OX) . . . 5

5 Multi-Vehicle: Capacitated Vehicle Routing Problem **6**
- 5.1 Problem Formulation . . . 6
- 5.2 Two-Phase Solution Algorithm . . . 7
  - 5.2.1 Phase 1: Mode-First Stop Assignment . . . 7
  - 5.2.2 Phase 2: Intra-Mode Distribution via k-Means . . . 7
  - 5.2.3 Per-Vehicle TSP . . . 8

6 Route Reconstruction **8**

7 Summary of Notation **8**

1 Overview

DeliveryIQ solves two related combinatorial optimization problems depending on the number of vehicles in the fleet:

1. **Single-vehicle mode** (fleet size = 1): the *Travelling Salesman Problem* (TSP) — find the shortest round-trip tour visiting all delivery stops exactly once and returning to the depot.
2. **Multi-vehicle mode** (fleet size ≥ 2): a *Capacitated Vehicle Routing Problem* (CVRP) — partition stops across a heterogeneous fleet of vehicles with per-vehicle transport mode and stop-capacity constraints, then solve a TSP for each vehicle's assigned cluster.

Both problems operate on a *weighted directed graph* extracted from OpenStreetMap (OSM) via the OSMnx library. Travel times along graph edges encode modal accessibility rules (drive / bike / walk) and serve as the optimization objective.

2 Road-Network Graph Model

2.1 Directed Weighted Graph

The street network is modelled as a directed multigraph

$$G = (V,\, E,\, w),$$

where:

- V is the set of OSM *nodes* (road intersections and endpoints),
- $E \subseteq V \times V$ is the set of directed *edges* (road segments),
- $w : E \to \mathbb{R}_{>0} \cup \{\mathcal{P}\}$ is the *travel-time weight function*, mapping each directed edge to a travel time in seconds or to the penalty sentinel $\mathcal{P} = 10^9$ (impassable edge).

2.2 Largest Strongly Connected Component (LSCC)

After download, the graph is pruned to its *Largest Strongly Connected Component*:

$$G^* = \mathrm{LSCC}(G),$$

ensuring that for every pair of nodes $u, v \in V^*$ a directed path from u to v exists within G^*. All subsequent computations use G^* exclusively.

2.3 Modal Travel-Time Stamping

Three independent copies of G^* are built, one per transport mode $m \in \mathcal{M} = \{\texttt{drive}, \texttt{bike}, \texttt{walk}\}$. The travel-time weight of edge (u, v) under mode m is:

$$w_m(u,v) = \begin{cases} \dfrac{\ell(u,v)}{s_m} & \text{if mode } m \text{ may legally traverse } (u,v), \\ \dfrac{\ell(u,v)}{s_m \cdot \alpha_{\text{steps}}} & \text{if } m = \texttt{walk} \text{ and } \texttt{highway} = \texttt{steps}, \\ \mathcal{P} & \text{otherwise (mode-impassable edge),} \end{cases} \tag{1}$$

where:

- $\ell(u,v) \geq 1$ is the physical length of the edge in metres,
- s_m is the assumed constant travel speed in metres per second (Table 1),
- $\alpha_{\text{steps}} = 0.5$ is the speed-reduction factor for stairs.

Table 1: Assumed travel speeds by mode.

Mode	Speed (km/h)	Speed (m/s)
Drive (`drive`)	30.0	$8.\overline{3}$
Bicycle (`bike`)	15.0	$4.1\overline{6}$
Walking (`walk`)	5.0	$1.3\overline{8}$

The modal accessibility rules applied in (1) are summarised in Table 2.

Table 2: Edge accessibility rules (PENALTY = impassable, else passable).

Edge type	`drive`	`bike`
`highway` $\in$ {footway, pedestrian, steps, ...}	$\mathcal{P}$	passable
`motor_vehicle` $\in$ {no, private, destination}	$\mathcal{P}$	passable
`bicycle` $\in$ {no, dismount}	n/a	$\mathcal{P}$
`highway` = cycleway	passable	passable
One-way reversed edge (no bike contra-flow tag)	—	$\mathcal{P}$
`foot` $\in$ {no, private}	n/a	n/a

3 Shortest-Path Distance Matrix

3.1 All-Pairs Shortest Paths (Dijkstra)

Given the modal graph $G_m = (V^*, E, w_m)$ and a set of locations $L = \{0, 1, \ldots, n\}$ where node 0 is the *depot* and nodes $1, \ldots, n$ are delivery stops, the *distance matrix* is defined as:

$$d_m(i,j) = \begin{cases} 0 & \text{if } i = j, \\ \min\limits_{\text{path } p: i \to j} \sum\limits_{(u,v) \in p} w_m(u,v) & \text{if a finite-cost path exists,} \\ \mathcal{P} & \text{otherwise.} \end{cases} \tag{2}$$

The minimisation in (2) is computed by Dijkstra's algorithm for every ordered pair (i, j) with $i \neq j$:

$$d_m(i, j) = \text{Dijkstra}(G_m,\ i,\ j,\ w_m).$$

Building the full matrix requires $n(n+1)$ Dijkstra calls ($(n+1)$ nodes, $n(n+1)$ ordered pairs).

3.2 Penalty Sentinel

Any pair (i, j) with $d_m(i, j) \geq \mathcal{P}$ is considered *mode-unreachable*. The value $\mathcal{P} = 10^9$ seconds (≈ 31.7 years) is chosen to dominate any realistic travel time while remaining a finite float, so TSP solvers can still construct a valid (if suboptimal) tour when unreachable stops are present.

3.3 Hybrid Last-Meter (Drive Mode)

For drive-mode routing, a stop j may lie in a pedestrian zone unreachable by car. In this case the car parks at the nearest car-accessible node $v_j^{\text{car}} \in V^*$ and the driver walks the remaining distance. The adjusted drive cost to stop j from stop i is:

$$d_{\text{hybrid}}(i, j) = d_{\text{drive}}(i,\ v_j^{\text{car}}) + \frac{\ell(v_j^{\text{car}},\ j)}{s_{\text{walk}}}, \tag{3}$$

where $\ell(v_j^{\text{car}}, j)$ is the straight-line (Haversine) distance from the parking node to the actual stop location.

4 Single-Vehicle: Travelling Salesman Problem

4.1 Problem Formulation

Let $L = \{0, 1, \ldots, n\}$ with depot 0. A *tour* is a permutation π of $\{1, \ldots, n\}$ that specifies the visit order. The total round-trip cost is:

$$C(\pi) = d(0,\ \pi_1) + \sum_{k=1}^{n-1} d(\pi_k,\ \pi_{k+1}) + d(\pi_n,\ 0), \tag{4}$$

where $d = d_m$ is the distance matrix for the chosen mode m.

The TSP seeks a permutation π^* minimising the tour cost:

$$\pi^* = \arg\min_{\pi \in \Pi_n} C(\pi), \tag{5}$$

where Π_n is the set of all permutations of n stops.

The tour is encoded as a *closed sequence* starting and ending at the depot:

$$[0,\ \pi_1,\ \pi_2,\ \ldots,\ \pi_n,\ 0].$$

4.2 Solution Methods

The TSP is NP-hard in general. The system selects a heuristic or approximation algorithm automatically based on the number of stops n (Table 3).

Table 3: Automatic TSP method selection.

Stop count n	Method
1–2	Nearest Neighbour (NN)
3–20	NN + 2-opt
21+	Genetic Algorithm (OX)
any	Christofides (explicit)

4.2.1 Nearest Neighbour (NN)

Starting from the depot, greedily append the closest unvisited stop at each step:

$$\pi_{k+1} = \underset{j \notin \{\pi_1,\ldots,\pi_k\}}{\arg\min} \; d(\pi_k,\, j), \qquad \pi_0 = 0. \tag{6}$$

The NN heuristic runs in $O(n^2)$ time and produces a feasible tour even when some matrix entries carry the penalty $\mathcal{P}$.

4.2.2 2-opt Local Search

Starting from the NN tour, iteratively swap two non-adjacent edges (r_i, r_{i+1}) and (r_j, r_{j+1}) if the swap decreases tour cost. For a tour $r = [r_0, r_1, \ldots, r_n, r_0]$, the *2-opt swap* of positions i and j reverses the sub-sequence $r_i, \ldots, r_j$:

$$r' = [r_0, \ldots, r_{i-1},\; r_j, r_{j-1}, \ldots, r_i,\; r_{j+1}, \ldots, r_n, r_0].$$

The swap is accepted when:

$$C(r') < C(r). \tag{7}$$

The procedure repeats until no improving swap exists or a maximum of 2 000 iterations is reached. Complexity per iteration pass: $O(n^2)$.

4.2.3 Christofides Approximation

The Christofides algorithm gives a $\frac{3}{2}$-approximation for the metric TSP. It requires a *complete, connected, undirected* helper graph $H = (L, E_H)$ where E_H contains only edges with $d(i, j) < \mathcal{P}$. If H is disconnected (i.e. some stop is unreachable), the algorithm falls back to 2-opt. When applicable, it proceeds as follows:

1. Compute a minimum spanning tree T of H.
2. Find the set $O \subset L$ of nodes with odd degree in T.
3. Compute a minimum-weight perfect matching M on the subgraph induced by O.
4. Form an Eulerian multigraph $H' = T \cup M$ and extract an Eulerian circuit.
5. Shortcut repeated vertices to obtain a Hamiltonian tour.

4.2.4 Genetic Algorithm (Order-Crossover, OX)

For large instances ($n \geq 21$), an evolutionary algorithm is used. The *chromosome* is a permutation π of stops $\{1, \ldots, n\}$ (depot excluded). The *fitness* function is:

$$f(\pi) = \frac{1}{C(\pi) + 1}, \tag{8}$$

so lower tour cost yields higher fitness.

Order crossover (OX). Given parents p_1 and p_2, select a random sub-sequence slice $[a, b)$:

1. Copy $p_1[a : b]$ directly into the child at positions $[a, b)$.
2. Fill remaining positions left-to-right with genes from p_2 in their original order, skipping genes already placed.

Swap mutation. With probability $\mu = 0.02$, exchange two randomly chosen genes: $\pi_i \leftrightarrow \pi_j$.

Generational loop. With population size $N_{\text{pop}} = 120$ and $G_{\max} = 400$ generations:

1. Sort population by fitness (descending).
2. Retain the top 10% as elites.
3. Fill the rest via OX crossover of randomly chosen elite parents, followed by swap mutation.

The best chromosome after $G_{\max}$ generations is returned as the approximate solution.

5 Multi-Vehicle: Capacitated Vehicle Routing Problem

5.1 Problem Formulation

Let:

- $S = \{s_1, \ldots, s_n\}$ be the set of delivery stops,
- $\mathcal{V} = \{v_1, \ldots, v_K\}$ be the fleet of K vehicles,
- $m_k \in \mathcal{M}$ be the transport mode of vehicle v_k,
- $Q_k \in \mathbb{Z}_{>0}$ be the stop capacity of vehicle v_k,
- $d_0 \in V^*$ be the depot node.

The CVRP seeks an assignment $\sigma : S \to \mathcal{V}$ and, for each vehicle v_k, a tour $\pi^{(k)}$ over the stops assigned to it, minimising the total fleet travel time:

$$\min_{\sigma, \{\pi^{(k)}\}} \sum_{k=1}^{K} C_k\left(\pi^{(k)}\right) \tag{9}$$

subject to:

$$\sigma(s_i) \neq \varnothing \implies d_{m_{\sigma(s_i)}}(d_0, s_i) < \mathcal{P} \qquad \text{(mode-compatibility)}, \tag{10}$$

$$\left|\sigma^{-1}(v_k)\right| \leq Q_k \qquad \forall\, k \in \{1, \ldots, K\} \quad \text{(capacity)}, \tag{11}$$

$$\sigma^{-1}(v_k) \cap \sigma^{-1}(v_j) = \varnothing \qquad \forall\, k \neq j \quad \text{(each stop assigned once)}, \tag{12}$$

where $C_k(\pi^{(k)})$ is the tour cost (4) for vehicle v_k using distance matrix d_{m_k}.

Stops unreachable by any fleet vehicle ($d_m(d_0, s_i) \geq \mathcal{P}$ for all $m \in \{m_k\}$) are excluded from the optimization and reported as warnings.

5.2 Two-Phase Solution Algorithm

The CVRP is solved via a *cluster-first, route-second* decomposition in two phases.

5.2.1 Phase 1: Mode-First Stop Assignment

Each stop s_i is assigned to the transport mode $m^*(s_i)$ whose vehicles currently have the most remaining total capacity among compatible modes:

$$m^*(s_i) = \arg\max_{m \in \mathcal{M}(s_i)} R_m, \qquad R_m = \sum_{k:\, m_k = m} \Big(Q_k - |\sigma^{-1}(v_k)|\Big), \tag{13}$$

where $\mathcal{M}(s_i) = \{m \in \mathcal{M} : d_m(d_0, s_i) < \mathcal{P}\}$ is the set of modes compatible with stop s_i, and R_m is the total remaining capacity across all vehicles of mode m.

After mode assignment, the stops are partitioned into mode-specific pools: $\mathcal{S}_m = \{s_i : m^*(s_i) = m\}$.

5.2.2 Phase 2: Intra-Mode Distribution via k-Means

Within each mode pool $\mathcal{S}_m$, stops are distributed across the $K_m = |\{k : m_k = m\}|$ vehicles of that mode using geographic k-means clustering.

k-Means clustering. Let $K'_m = \min(K_m, |\mathcal{S}_m|)$. Run k-means on the geographic coordinates of the stops in $\mathcal{S}_m$:

$$\min_{\{C_1, \ldots, C_{K'_m}\}} \sum_{j=1}^{K'_m} \sum_{s_i \in C_j} \left\| \begin{pmatrix} \varphi(s_i) \\ \lambda(s_i) \end{pmatrix} - \boldsymbol{\mu}_j \right\|^2, \tag{14}$$

where $\varphi(s_i)$ and $\lambda(s_i)$ are the latitude and longitude of stop s_i, $\boldsymbol{\mu}_j$ is the centroid of cluster j, and $\|\cdot\|$ denotes the Euclidean norm.

Bearing-based cluster-to-vehicle assignment. To assign clusters to vehicles in a geographically coherent way, cluster centroids are sorted by their *compass bearing* from the depot. The bearing from point (φ_1, λ_1) to (φ_2, λ_2) is:

$$\theta = \left(\arctan 2(\sin(\Delta\lambda)\cos\varphi_2,\ \cos\varphi_1 \sin\varphi_2 - \sin\varphi_1 \cos\varphi_2 \cos(\Delta\lambda)) \cdot \frac{180}{\pi} + 360\right) \bmod 360, \tag{15}$$

where $\Delta\lambda = \lambda_2 - \lambda_1$. Clusters are assigned to vehicles in ascending bearing order.

Capacity rebalancing. k-means ignores vehicle capacities. After the initial assignment, any vehicle v_k that exceeds its capacity Q_k transfers excess stops to the next vehicle with remaining capacity:

$$\text{while } |\sigma^{-1}(v_k)| > Q_k : \quad \text{pop last stop from } \sigma^{-1}(v_k), \quad \text{insert into first } v_j \text{ with } |\sigma^{-1}(v_j)| < Q_j. \tag{16}$$

Candidate vehicles are scanned first forward ($j > k$) then backward ($j < k$) to avoid false over-capacity errors on the last vehicle in bearing order. If no vehicle has remaining capacity a `ValueError` is raised.

5.2.3 Per-Vehicle TSP

Once the assignment σ is determined, each vehicle v_k with $|\sigma^{-1}(v_k)| \geq 1$ independently solves the TSP (5)–(4) on its cluster using the modal distance matrix d_{m_k} and the auto-selected method from Table 3.

The total fleet cost decomposes as:

$$C_{\text{fleet}} = \sum_{k=1}^{K} C_k(\pi^{(k)}). \tag{17}$$

6 Route Reconstruction

The TSP solution π^* is a sequence of high-level location nodes $[0, \pi_1^*, \ldots, \pi_n^*, 0]$. To render the route on the map, each consecutive pair (r_k, r_{k+1}) is expanded to the full sequence of intermediate OSM road nodes via Dijkstra:

$$\text{path}(r_k, r_{k+1}) = \underset{p:\, r_k \to r_{k+1}}{\arg\min} \sum_{(u,v) \in p} w_m(u, v). \tag{18}$$

The concatenation of all expanded sub-paths (with shared boundary nodes deduplicated) forms the *full route polyline* drawn on the Folium map.

7 Summary of Notation

Table 4: Summary of mathematical notation.

Symbol	Description
$G = (V, E, w)$	Directed weighted multigraph of the road network
G^*	Largest Strongly Connected Component of G
G_m	Modal copy of G^* for mode m
$w_m(u, v)$	Travel-time weight of edge (u, v) under mode m (seconds)
$\mathcal{P} = 10^9$	Impassable / unreachable sentinel value (seconds)
$\ell(u, v)$	Physical length of edge (u, v) in metres
s_m	Travel speed for mode m in metres per second
$d_m(i, j)$	Shortest-path travel time from i to j under mode m
n	Number of delivery stops (excluding depot)
$L = \{0, 1, \ldots, n\}$	Set of locations; node 0 is the depot
π	Permutation of stops (TSP tour, excluding depot)
$C(\pi)$	Total round-trip tour cost
π^*	Optimal (or near-optimal) tour
K	Number of vehicles in the fleet
$\mathcal{V} = \{v_1, \ldots, v_K\}$	Fleet of vehicles
m_k	Transport mode of vehicle v_k
Q_k	Stop capacity of vehicle v_k
$\sigma : S \to \mathcal{V}$	Stop-to-vehicle assignment function
$\mathcal{S}_m$	Set of stops assigned to mode pool m
C_j	j-th k-means cluster of stops
$\boldsymbol{\mu}_j$	Centroid of cluster C_j
θ	Compass bearing (degrees, $[0, 360)$)
$f(\pi)$	Genetic algorithm fitness (reciprocal cost)